

Word-Level Plagiarism Checker for Short Documents

L.Tejasri-2520090017
S.Dikshita-2520090007

ABSTRACT

- **The Word-Level Plagiarism Checker for Short Documents is a plagiarism detection system designed to identify copied and lightly paraphrased content in short text documents such as essays, abstracts, reports, and assignment answers.**
- **The project compares an input document with a reference corpus at the word level using custom-built data structures instead of prebuilt library implementations.**
- **It detects exact matches, reordered sentences, and reworded passages by analyzing word sequences and similarity patterns.**
- **The system also identifies the original source of each plagiarized section and highlights the matching text for easy review, making the results clear and easy to understand. Optimized for fast processing, it delivers results suitable for interactive use by instructors, editors, publishers, and students.**
- **By combining accuracy, efficiency, detailed source attribution, and user-friendly output, the project provides a reliable solution for plagiarism detection while demonstrating the practical implementation of fundamental data structures and algorithms in a real-world application.**

INTRODUCTION

Lorem ipsum dolor sit amet, consectetur

Plagiarism is a common problem in academic and professional documents. This project develops a Word-Level Plagiarism Checker that compares a document with reference texts to identify copied and lightly modified content. It analyzes the similarity between documents at the word level and highlights matching or suspicious sections for easy identification. The system can help students, researchers, and professionals maintain originality by providing a clear plagiarism percentage and detailed comparison results.

SOFTWARE REQUIREMENTS

- **Operating System: Windows / Linux**
- **Programming Language: Java**
- **JDK: Java Development Kit (JDK)**
- **IDE/Code Editor: Visual Studio Code / IntelliJ IDEA / Eclipse**
- **Core Algorithms:**
 - **Naïve Pattern Matching**
 - **KMP Algorithm**
 - **Rabin-Karp with Rolling Hash**
 - **Suffix Arrays and LCP**
 - **Edit Distance using Wagner-Fischer**
 - **Dynamic Programming**
- **Data Handling: Text files / document corpus**
- **Algorithm Implementation: Custom-built data structures and algorithms**

Hardware Requirements

- **Computer/Laptop**
- **Processor: Intel/AMD processor**
- **RAM: Minimum 4 GB**
- **Storage: Minimum 10 GB free space**
- **Keyboard and Mouse**
- **Monitor/Display**

METHODOLOGY

- 1.Input Document:** The user provides a short document that needs to be checked.
- 2.Preprocessing:** The document is divided into words and sentences, and unnecessary elements are removed.
- 3.Reference Corpus:** The system stores or reads reference documents that are used for comparison.
- 4.Word-Level Comparison:** Words and word sequences from the input document are compared with the reference corpus.
- 5.Similarity Detection:** The system detects exact matches, reordered sentences, and lightly reworded passages.
- 6.Source Identification:** For each detected matching section, the system identifies the corresponding original source.
- 7.Highlighting:** Matching or potentially plagiarized portions are highlighted for easier review.
- 8.Result Generation:** The system presents the detected similarities and source information in a clear format.
- 9.Optimization:** Custom data structures and algorithms are used to improve processing speed and demonstrate DSA concepts.

Literature Survey

- Existing plagiarism detection methods use string matching and text similarity techniques to identify copied content.
- Algorithms such as KMP, Rabin-Karp, and suffix arrays help in efficient text searching and comparison.
- Edit distance techniques can identify differences between original and modified text.
- These approaches are useful for detecting both exact matches and lightly paraphrased content.

THANK YOU